

EXAMEN PARCIAL 2 — BASES DE DATOS II

Alumno: Patricio Sussini

Materia: Bases de Datos II

Carrera: Tecnicatura Universitaria en Programación a Distancia

1. Enunciado

Usted es el administrador de la base de datos de la plataforma de e-commerce **tiendaDB**, cuya colección principal es **productos**. Su tarea es garantizar la correcta gestión de datos, optimizar el rendimiento de consultas y aplicar buenas prácticas operativas.

Sección 1 — Gestión de datos y consola

Indicar comandos de *mongosh* y operaciones CRUD esenciales para manipular documentos y mantener su integridad.

1. Creación e Inserción

- Indique el comando necesario para cambiar a la base de datos **tiendaDB** desde la consola *mongosh*.
- Escriba la operación que inserta *en una sola ejecución* los siguientes dos productos en la colección **productos**: Producto A (nombre: "Teclado Mecánico Redragon", precio: 65000, stock: 15, tags: ["gaming", "perifericos"]) y Producto B (nombre: "Mouse Inalámbrico Logitech", precio: 20000, stock: 30, tags: ["perifericos", "oficina"]).

2. Actualización de Arrays y Campos

- El producto "Teclado Mecánico Redragon" está por agotarse. Indique el comando que reduce su stock a 7 unidades modificando únicamente ese campo.
- Escriba el comando para agregar el tag "RGB" a su array tags, evitando duplicados.
- Escriba el comando que elimina el tag "oficina" del array tags del producto "Mouse Inalámbrico Logitech".

Sección 2 — Optimización de Consultas

Crear y evaluar índices que mejoren el rendimiento en consultas frecuentes.

1. Creación y Evaluación de Índices

- Indique el comando para crear un índice simple en orden ascendente sobre el campo precio.
- La empresa necesita consultar por nombre y luego ordenar por precio. Indique el índice compuesto que optimiza esta consulta (principio de index prefixing).
- Al ejecutar `explain("executionStats")` sobre la consulta `db.productos.find({ tags: "gaming" })`, ¿qué valor debería verse en `winningPlan.stage` para confirmar que el índice fue utilizado correctamente?

2. Agregación — \$group

Escriba un pipeline de agregación que devuelva el stock total de todos los productos y el precio promedio general.

Sección 3 — Gestión Operacional

Aplicar comandos de terminal y comprender estrategias de backup.

1. Comandos de Terminal (CLI)

- Desde el directorio actual `/home/usuario/Proyectos`, indique el comando relativo para acceder a `/home/usuario/Documentos`.
- La carpeta `backups_temp/` debe ser eliminada junto con todo su contenido. Indique el comando de Linux para eliminarla de manera recursiva y el comando equivalente en Windows (CMD).

2. Estrategias de Backup

- Una empresa realiza Backup Completo los domingos y Backups Diferenciales de lunes a sábado. Si ocurre un fallo un jueves por la noche, ¿cuántos archivos de respaldo deben aplicarse para restaurar el estado más reciente? Justifique.
 - Explique la diferencia fundamental entre RPO (Recovery Point Objective) y RTO (Recovery Time Objective) en el contexto de continuidad operativa.
-

2. Resolución

Sección 1 — Gestión de datos y consola

1. Creación e Inserción

a) Cambiar a la base de datos `tiendaDB`

En mongosh, el comando **`use`** selecciona la base de datos indicada. Si no existe, MongoDB la crea automáticamente en el momento en que se inserta el primer documento.

```
use tiendaDB
```

b) Inserción de dos productos en una sola ejecución

Para insertar múltiples documentos en una sola operación se utiliza **`insertMany()`**, que recibe un array de documentos y los inserta de forma atómica por lote, reduciendo el overhead de red respecto a múltiples llamadas a `insertOne()`.

```
db.productos.insertMany([
  {
    nombre: "Teclado Mecanico Redragon",
    precio: 65000,
    stock: 15,
    tags: ["gaming", "perifericos"]
  },
  {
    nombre: "Mouse Inalambrico Logitech",
    precio: 20000,
    stock: 30,
    tags: ["perifericos", "oficina"]
  }
])
```

2. Actualización de Arrays y Campos

a) Reducir el stock del Teclado Mecánico Redragon a 7 unidades

Se utiliza el operador **`$set`** para modificar únicamente el campo `stock` sin alterar el resto del documento. El filtro por nombre identifica de forma unívoca el producto a actualizar.

```
db.productos.updateOne(
```

```

    { nombre: "Teclado Mecanico Redragon" },
    { $set: { stock: 7 } }
  )

```

b) Agregar el tag "RGB" evitando duplicados

El operador **\$addToSet** agrega un valor a un array únicamente si ese valor no existe ya en él, garantizando que el array no contenga duplicados. A diferencia de \$push, \$addToSet verifica la existencia del elemento antes de insertarlo.

```

db.productos.updateOne(
  { nombre: "Teclado Mecanico Redragon" },
  { $addToSet: { tags: "RGB" } }
)

```

c) Eliminar el tag "oficina" del Mouse Inalámbrico Logitech

El operador **\$pull** elimina del array todos los elementos que coincidan con el valor o condición especificada. En este caso remueve el string "oficina" del campo tags.

```

db.productos.updateOne(
  { nombre: "Mouse Inalambrico Logitech" },
  { $pull: { tags: "oficina" } }
)

```

Sección 2 — Optimización de Consultas

1. Creación y Evaluación de Índices

a) Índice simple sobre el campo precio (ascendente)

El valor **1** indica orden ascendente. Este índice mejora las consultas que filtran o ordenan por precio, evitando un COLLSCAN (escaneo completo de la colección).

```

db.productos.createIndex({ precio: 1 })

```

b) Índice compuesto para consultar por nombre y ordenar por precio

El principio de **index prefixing** establece que el campo utilizado en el filtro debe ser el prefijo (primera clave) del índice compuesto, de modo que MongoDB pueda usarlo tanto para resolver el filtro como para ordenar sin un paso adicional de sort en memoria. Por ello, *nombre* va primero y *precio* segundo.

```

db.productos.createIndex({ nombre: 1, precio: 1 })

```

c) Valor esperado en winningPlan.stage

Al ejecutar **explain("executionStats")** sobre la consulta, el valor esperado en *winningPlan.stage* es:

```

IXSCAN

```

Este valor indica que MongoDB realizó un *Index Scan*, es decir, recorrió el índice en lugar de hacer un escaneo completo de la colección (COLLSCAN). La presencia de IXSCAN confirma que el índice fue reconocido y utilizado correctamente por el query planner.

2. Agregación — \$group

El pipeline utiliza una sola etapa **\$group** con *_id: null* para agrupar todos los documentos en un único resultado. El operador **\$sum** acumula el campo stock y **\$avg** calcula el promedio del campo precio sobre toda la colección.

```

db.productos.aggregate([
  {

```

```

    $group: {
      _id: null,
      stockTotal: { $sum: "$stock" },
      precioPromedio: { $avg: "$precio" }
    }
  }
]

```

Sección 3 — Gestión Operacional

1. Comandos de Terminal (CLI)

a) Ruta relativa desde /home/usuario/Proyectos hacia /home/usuario/Documentos

Ambas carpetas se encuentran al mismo nivel bajo /home/usuario/. Para navegar de una a otra de forma relativa se sube un nivel con `..` y luego se entra al directorio destino:

```
cd ../Documentos
```

b) Eliminación recursiva de la carpeta backups_temp/

Linux (Bash):

```
rm -rf backups_temp/
```

Las flags `-r` (recursive) y `-f` (force) eliminan el directorio y todo su contenido sin solicitar confirmación.

Windows (CMD):

```
rmdir /s /q backups_temp
```

El flag `/s` indica eliminación recursiva de subdirectorios y archivos, y `/q` suprime el mensaje de confirmación (modo silencioso).

2. Estrategias de Backup

a) Archivos de respaldo necesarios para restaurar tras un fallo el jueves por la noche

La estrategia combina un **Backup Completo** semanal (domingo) con **Backups Diferenciales** diarios (lunes a sábado). Un backup diferencial almacena *todos los cambios acumulados desde el último backup completo*, a diferencia del incremental que solo guarda los cambios desde el último backup (sea completo o incremental).

Por lo tanto, el backup diferencial del jueves ya contiene los cambios del lunes, martes, miércoles y jueves desde el domingo. Para restaurar al estado más reciente se necesitan **exactamente 2 archivos**:

- **1** — Backup Completo del domingo (base de restauración).
- **2** — Backup Diferencial del jueves (todos los cambios desde el domingo).

Esta es precisamente la ventaja del esquema diferencial sobre el incremental: la restauración siempre requiere solo dos archivos (completo + último diferencial), independientemente de cuántos días hayan transcurrido desde el último backup completo.

b) Diferencia entre RPO y RTO

Ambos conceptos son métricas clave de la continuidad operativa, pero miden dimensiones distintas del riesgo:

RPO (Recovery Point Objective) — Tolerancia a la pérdida de datos: Define cuánto tiempo hacia atrás se puede retroceder en los datos sin que el negocio sufra un impacto inaceptable. En otras palabras, es la cantidad máxima de datos (medida en tiempo) que la organización acepta perder en caso de fallo. Un

RPO de 24 horas significa que si ocurre un desastre, se puede tolerar perder hasta un día de transacciones.

RTO (Recovery Time Objective) — Tolerancia al tiempo de inactividad: Define el tiempo máximo tolerable que el sistema puede estar fuera de servicio desde el momento del fallo hasta que queda completamente restaurado y operativo. Un RTO de 4 horas significa que el sistema debe estar funcionando de nuevo dentro de las 4 horas posteriores al incidente.

En síntesis: el **RPO mide cuánto pasado se puede perder** (datos), mientras que el **RTO mide cuánto futuro se puede tolerar sin servicio** (tiempo de recuperación). Ambos objetivos guían las decisiones sobre la frecuencia de backups, la arquitectura de alta disponibilidad y el presupuesto destinado a la continuidad del negocio.